Unit 07: Run-time Polymorphism

CONTENTS

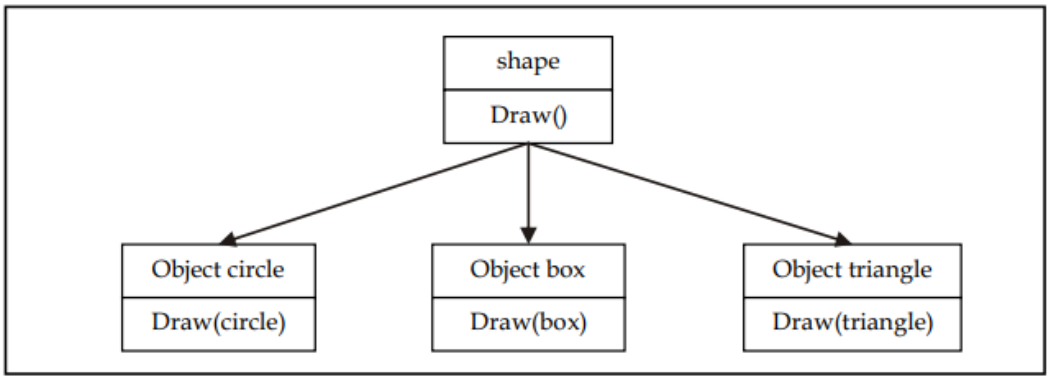
Objectives

After studying this unit, you will be able to:

- Understand run time polymorphism
- Describe the virtual base class, abstract class
- Recognize pointer to object
- Explain this pointer
- Analyze pointer to derived class

Introduction

Polymorphism is a key concept in OOP. Polymorphism refers to the ability to take on multiple identities. An operation, for example, may behave differently in different situations. The behavior is determined by the data types utilized in the operation. Take, for example, the addition operation. For tow numbers, the operation will generate a sum. If the operands are strings, then the operation will produce a third string by contention. The diagram given below, illustrates that a single function name can be used to handle different number and types of arguments. This is something similar to a particular word having several different meanings depending on the context. Polymorphism is crucial in allowing things with disparate internal structures to share the same external interface. This means that even while the precise actions associated with each operation may differ, a generic class of operations can be accessed in the same way. As demonstrated here, polymorphism is widely used to implement inheritance.



Polymorphism can be implemented using operator and function overloading, where the same operator and function works differently on different arguments producing different results. These polymorphisms are brought into effect at compile time itself, hence is known as early binding, static binding, static linking or compile time polymorphism.

However, uncertainty arises when a function with the same name exists in both the base and derived classes. Consider the following code snippet as an example.

```
Class ab
{
Int a;
Public:
Void display();    {……}                    //        display in base class
};
Class ba: public ab
{
Int b;
Public:
Void display();    {…….}                   //        display in derived class
};
```

There is no overloading because the functions ab.display() and ba.display() are the same but in distinct classes, therefore early binding does not apply. Run time polymorphism selects the right function at run time.

7.1 Run-time Polymorphism

C++ supports run-time polymorphism by a mechanism called virtual function. It exhibits late binding or dynamic linking.

Caution: What do you mean by function overriding?

Function overriding occurs when a child class declares a method that already exists in the parent class; in this case, the child class overrides the parent class. Function overriding is an example of Runtime polymorphism.

Example

```
// Program of runtime polymorphism


#include <iostream>
```

```
using namespace std;
class A {
public:
  void disp(){
    cout<<"Display in base class"<<endl;
  }
};
class B: public A{
public:
  void disp(){
    cout<<"Display in derived class";
  }
};
int main() {
  //Parent class object
  A obj;
  obj.disp();
  //Child class object
  B obj2;
  obj2.disp();
  return 0;
}
```

Output

```
Display in base class
Display in derived class
Process returned 0 (0x0)   execution time : 0.356 s
Press any key to continue.
```

7.2 Virtual Base Class

Virtual classes are primarily used during multiple inheritance. Virtual base class is used in situation where a derived have multiple copies of base class. When two or more objects are derived from a common base class, we can prevent multiple copies of the base class being present in an object derived from those objects by declaring the base class as virtual when it is being inherited.

Virtual base class means that child classes can access the base classes virtually from a long distance in the parent-child hierarchy.

 Lab Exercise

```
#include<iostream>
using namespace std;
class A{
```

```
public:
   int x;
};
class B:virtual public A{
public:
   int y;
};
class C: virtual public A{
public:
   int z;
};
class D:public C,public B{
public:
   int x1;
};
main(){
D obj;
obj.x=100;
obj.y=20;
obj.z=30;
obj.x1=200;
cout<< "\n X : "<< obj.x;
            cout<< "\n Y : "<< obj.y;
        cout<< "\n Z : "<< obj.z;
        cout<< "\n X1 : "<<              obj.x1;
            }
```

Output

```
 X : 100
 Y : 20
 Z : 30
 X1 : 200
Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.
```

7.3 Abstract Class

An abstract class in C++ has at least one pure virtual function by definition. In other words, a function that has no definition. The abstract class's descendants must define the pure virtual function; otherwise, the subclass would become an abstract class in its own right.

Abstract classes are used to describe general notions that can be used to create more concrete classes. It is not possible to generate an abstract class type object. However, pointers and references can be used to abstract class types. When building an abstract class, make sure to include at least one pure virtual member feature. A virtual function is declared using the pure specifier (= 0) syntax.

Notes

- A class that contains a pure virtual function is known as an abstract class.
- Abstract classes are essential to providing an abstraction to the code to make it reusable and extendable.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.

Properties of Abstract Class

1. Abstract classes cannot have objects.
2. To be an abstract class, it must have a presence of at least one virtual class.
3. We can use pointers and references to abstract class type.
4. We can create constructors of an abstract class.

Did you know?

Pure virtual Functions are virtual functions with no definition. They start with virtual keyword and ends with = 0.

Lab Exercise

```
// Program
#include <iostream>
using namespace std;
class A
{
public:
  virtual void test() = 0;
};
```

```
class B : public A
{
public:
  void test()
   {
cout << " Hello !  Virtual function " << endl;
  }
};
int main(void)
{
  B obj;
 obj.test();
 return 0;
}
```

Output

```
 Hello !  Virtual function

Process returned 0 (0x0)   execution time : 0.131 s
Press any key to continue.
```

Task: Write a Program to demonstrate working of abstract class in C++.

Restriction of Abstract Class

Abstract classes cannot be used for the following scenarios:

- Member data or variables
- Forms of debate
- Types of function output
- Conversions that are made explicitly

7.4 Pointer to object in C++

In C++ a pointer to a class is created in the same way as a pointer to a structure, and you use the member access operator -> to access members of a pointer to a class, just as you do with pointers to structures. You must also initialize the pointer before using it, as with all pointers.

Lab Exercise

```
//Program
#include <iostream>
using namespace std;
class sum
```

```
{
int b;
public:
void get_data(int a ){
b=a;
}
int display(){
return b;
}
};
main(){
sum obj;
sum* p;
p=&obj;
p->get_data(10);
cout<<" Value of a "<<p->display();
}
```

Output

```
 Value of a 10
Process returned 0 (0x0)   execution time : 0.177 s
Press any key to continue.
```

Caution: C++ class is done exactly the same way as a pointer to a structure and to access members of a pointer to a class access operator (arrow operator) -> is used.

7.5 The 'this' pointer

The 'this' pointer is also another concept here in which in C++ programming, this is a keyword that refers to the current instance of the class. If we are going to execute the current instance of the class, that is the object of my class there and if we have the privilege there using this pointer, we can add the data into there and we can store some of the information according to that use of this keyword, it can be used to pass current object as the parameter to the another method, and it can be used to refer the current class instance variable like we have a class members and their member functions if we want to use the this pointer there we can access the current members of member functions with this pointer it can be used to declare the indexes.

Uses of 'this' keyword

- It can be used to pass current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.

Consider the following example

Class ABC

{

Int a;

…….

…….

};

The private variable 'a' can be used directly inside a member function, like

a=100;

We can also use the following statement to do the same job:

this->a=100;

Since C++ permits the use of shorthand form a 100, we have not been using the pointer this explicitly SO far. However, we have been implicitly using the pointer this when overloading the operators using member function.

 Lab Exercise

```
//Program
#include <iostream>
using namespace std;
class employee
{
int id;
string name;
int salary;
public:
employee(int e_id,string  e_name,int e_salary ){
this->id=e_id;
this->name=e_name;
this->salary=e_salary;
}
void display(){
cout<<id<<name<<salary<<endl;
}
};
main(){
employee e1=employee(10,"john",25000);
e1.display();
}
```

Output

```
10john25000

Process returned 0 (0x0)   execution time : 0.049 s
Press any key to continue.
```

Task:

- Write a program to demonstrate working of pointer to object in C++.
- Write a program to demonstrate working of 'this' pointer in C++.

7.6 Pointer to Derived Class

We can use pointers not only to the base objects but also to the objects of derived classes. Pointers to objects of a base class are type-compatible with pointers to objects of a derived class. Therefore, a single minter variable ran he made to paint to objects belonging to different classes. For example, if **B** is a base class and **D** is a derived class from B, then a pointer declared as a pointer to **B** can also be a pointer to **D**. Consider the following declarations:

```
B *cptr;                    //Pointer to class B type variable
B b;                        //Base object
D d;                        //Derived object
Cptr=&b;                    //Cptr points to object b
```

We can make cptr to point to the object d as follows:

```
Cptr=&d;                    //cptr points to object d
```

This is perfectly valid with C++ because d is an object derived from the class B.

However, there is a problem in using cptr to access the public members of the derived class D. Using cptr, we can access only those members which are inherited from B and not the members that originally belong to D. In case a member of D has the same name as one of the members of B, then any reference to that member by cptr will always access the base class member.

Although C++ permits a base pointer to point to any object derived from that base, the pointer cannot be directly used to access all the members of the derived class. We may have to use another pointer declared as pointer to the derived type.

Lab Exercise

```
//Program
#include<iostream>
using namespace std;
class base
{
   public:
    void show()
   {
      cout<<"Base class"<<endl;
```

```
    }
};
class derive : public base
{
    public:
     void display()
    {
 cout<<"Derived Class";
    }
};
int main()
{
   base b;
   base *bptr;
  bptr->show();
  derive d;
bptr=&d;
((derive *)bptr)->display();
   return 0;
}
```

Output

```
Base class
Derived Class
Process returned 0 (0x0)   execution time : 0.141 s
Press any key to continue.
```

Summary

- Polymorphism simply means one name having multiple forms.
- There are two types of polymorphism, namely, compile time polymorphism and run time polymorphism.
- Functions and operators overloading are examples of compile time polymorphism. The overloaded member functions are selected for invoking by matching arguments, both type and number. The compiler knows this information at the compile time and, therefore, compiler is able to select the appropriate function for a particular call at the compile time itself. This is called early or static binding or static linking. It means that an object is bound to its function call at compile time.
- In run time polymorphism, an appropriate member function is selected while the program is running. C++ supports run time polymorphism with the help of virtual functions. It is called late or dynamic binding because the appropriate function is selected dynamically at

run time. Dynamic binding requires use of pointers to objects and is one of the powerful features of C++.

- Object pointers are useful in creating objects at run time. It can be used to access the public members of an object, along with an arrow operator.
- A this pointer refers to an object that currently invokes a member function. For example, the function call ashow0 will set the pointer 'this' to the address of the object 'a'.
- Pointers to objects of a base class type are compatible with pointers to objects of a derived class. Therefore, we can use .1 single pointer variable to point to objects of base class as well as derived classes.
- When a function is made virtual, C-+ determines which function to use at run time based on the type of object pointed to by the base pointer, rather than the type of the pointer. By making the base pointer to point to different objects, we can execute different versions of the virtual function.
- Run time polymorphism is achieved only when a virtual function is accessed through a pointer to the base class. It cannot be achieved using object name along with the dot operator to access virtual function.

Keywords

This Pointer: The pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Abstract Class: A class that contains a pure virtual function is known as an abstract class.

Function Pointer: A function may return a reference or a pointer variable also. A pointer to a function is the address where the code for the function resides. Pointer to functions can be passed to functions, returned from functions, stored in arrays and assigned to other pointers.

Memory location: A container that can store a binary number.

Virtual Base Class: Virtual base class is used in situation where a derived have multiple copies of base class.

Pointer: A variable holding a memory address.

Alias: A different name for a variable of C++ data type. Base Address: Starting address of a memory location holding array elements.

Reference: An alias for a pointer that does not require de-referencing to use.

Self Assessment

1. Which class is used to design the base class?

A. abstract class

B. derived class

C. base class

D. derived & base class

2. Which is also called as abstract class?

A. virtual function

B. pure virtual function
C. derived class
D. base class

3. What will be the output of the following C++ code?

```
#include <iostream>
using namespace std;
class MyInterface
{
   public:
   virtual void Display() = 0;
};
class Class1 : public MyInterface
{
   public:
   void Display()
   {
      int  a = 5;
      cout << a;
   }
};
class Class2 : public MyInterface
{
   public:
   void Display()
   {
      cout <<" 5" << endl;
   }
};
int main()
{
   Class1 obj1;
   obj1.Display();
   Class2 obj2;
   obj2.Display();
   return 0;
}
```

A. 5
B. 10
C. 15
D. 5 5

4. What is meant by pure virtual function?
a. Function which does not have definition of its own
b. Function which does have definition of its own
c. Function which does not have any return type
d. Function which does not have any return type & own definition

5. Where the abstract class does is used?
A. base class only
B. derived class
C. both derived & base class
D. virtual class

6. Which of the following is the correct way to declare a pointer?
A. int *ptr
B. int ptr
C. int &ptr
D. All of the above

7. Which is the pointer which denotes the object calling the member function?
A. Variable pointer
B. This pointer
C. Null pointer
D. Zero pointer

8. The this pointer is accessible _____
A. Within all the member functions of the class
B. Only within functions returning void
C. Only within non-static functions
D. Within the member functions with zero arguments

9. What is the use of this pointer?
A. When local variable's name is same as member's name, we can access member using this pointer.
B. To return reference to the calling object
C. Can be used for chained function calls on an object
D. All of the above

10. This pointer can be used directly to _____
A. To manipulate self-referential data structures
B. To manipulate any reference to pointers to member functions
C. To manipulate class references

D. To manipulate and disable any use of pointers

11. Which members can never be accessed in derived class from the base class?

A. Private

B. Protected

C. Public

D. All except private

12. Which of the following is true for pointer to derived class?

A. We can use pointers not only to a base objects but also to the objects of derived classes.

B. We cannot use pointers in the classes.

C. Using pointer in class is form of inheritance

D. All of above.

13. We can access those members of derived class which are inherited from base class by base class pointer.

A. True

B. False

14. What will be the output of following code?

```
#include<iostream>
using namespace std;
class base
{
    public:
     void show()
    {
       cout<<"Base class"<<endl;
    }
};
class derive : public base
{
    public:
     void display()
    {
  cout<<"Derived Class";
    }
};
int main()
{
   base b;
```

```
 base *bptr;
 bptr->show();
 derive d;
bptr=&d;
((derive *)bptr)->display();
 return 0;
}
```

A. Base class Derived Class
B. Base Class
C. Derived Class
D. All of Above

15. If same message is passed to objects of several different classes and all of those can respond in a different way, what is this feature called?

A. Inheritance
B. Overloading
C. Polymorphism
D. Overriding

Answers for Self Assessment

1.	A	2.	B	3.	D	4.	A	5.	A
6.	A	7.	B	8.	C	9.	D	10.	A
11.	D	12.	A	13.	A	14.	A	15.	C

Review Questions

1. What does polymorphism mean in C++ language?
2. How is polymorphism achieved at run time? Explain with suitable example.
3. What does this pointer point to?
4. What are the applications of this pointer?
5. Write a program that demonstrate working of virtual base class.
6. Why do we need abstract class?
7. What is run time polymorphism?
8. Explain pointer to derived class with suitable example.
9. How does pointer variable differ from simple variable?
10. Write a program that demonstrate the working of abstract class.

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

Web Links

http://msdn.microsoft.com/en-us/library/wcz57btd(v=vs.80).aspx

http://www.learncpp.com/cpp-tutorial/117-multiple-inheritance/

https://www.codecademy.com/learn/learn-c-plus-plus